

6.1 Analiza certyfikatu SSL/TLS witryny.

- (a) Za pomocą narzędzia **OpenSSL**, pobierz certyfikat strony internetowej <https://github.com>.

```
echo | openssl s_client -servername github.com -connect github.com:443 2>/dev/null | openssl x509 -out github.crt
```

- (b) Wyświetl certyfikat w czytelnej postaci.

- (c) Przeanalizuj certyfikat i odczytaj z niego następujące informacje (używając narzędzia **OpenSSL**):

- Dla jakiej domeny wystawiony jest certyfikat?
- Kto wystawił certyfikat (CA)?
- Kiedy certyfikat został wystawiony i kiedy wygasa?
- Jakie dodatkowe domeny są obsługiwane?
- Jaki algorytm i długość klucza?
- Do czego może być użyty certyfikat?
- Ile certyfikatów znajduje się w łańcuchu zaufania?
- Kto jest głównym CA (root CA) w łańcuchu?

- (d) Za pomocą narzędzia **OpenSSL**, wyodrębnij z certyfikatu klucz publiczny i zapisz go do pliku.

6.2 Pobierz cały łańcuch certyfikatów z serwera <https://github.com> przy użyciu **OpenSSL** i sprawdź liczbę certyfikatów w łańcuchu. Następnie wskaż, który certyfikat należy do serwera, a które do CA. Użyj poniższego polecenia:

```
openssl s_client -connect github.com:443 -showcerts </dev/null > gh.txt 2>/dev/null
```

6.3 Analiza certyfikatu SSL/TLS lokalnego serwera (certyfikatu typu self-signed).

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8443:443 --name tls3 docker.io/mazurkatarzyna/tls-ex3:latest  
podman run -p 8443:443 --name tls3 docker.io/mazurkatarzyna/tls-ex3:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8443:443 --name tls3 ghcr.io/mazurkatarzynaumcs/tls-ex3:latest  
podman run -p 8443:443 --name tls3 ghcr.io/mazurkatarzynaumcs/tls-ex3:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8443>.

- (b) Wyświetl certyfikat w czytelnej postaci.

- (c) Przeanalizuj certyfikat i odczytaj z niego następujące informacje (używając narzędzia **OpenSSL**):

- Dla jakiej domeny wystawiony jest certyfikat?
- Kto wystawił certyfikat (CA)?
- Kiedy certyfikat został wystawiony i kiedy wygasa?
- Jakie dodatkowe domeny są obsługiwane?
- Jaki algorytm i długość klucza?

- Do czego może być użyty certyfikat?
- Ile certyfikatów znajduje się w łańcuchu zaufania?
- Kto jest głównym CA (root CA) w łańcuchu?

(d) Za pomocą narzędzia `OpenSSL`, wyodrębnij z certyfikatu klucz publiczny i zapisz go do pliku.

6.4 Pobierz cały łańcuch certyfikatów z serwera <https://127.0.0.1:8443> przy użyciu `OpenSSL` i sprawdź liczbę certyfikatów w łańcuchu. Następnie wskaż, który certyfikat należy do serwera, a które do CA. Użyj poniższego polecenia:

```
openssl s_client -connect 127.0.0.1:8443 -showcerts </dev/null > gh.txt 2>/dev/null
```

6.5 Pod adresem <https://127.0.0.1:8447> działa prosty serwer pozwalający przetestować swoją wiedzę odnośnie certyfikatów TLS. Serwer posiada 2 endpointy, <https://127.0.0.1:8447/cert>, pod którym zwraca losowy certyfikat, oraz <https://127.0.0.1:8447/validate>, pod który można wysłać swoje odpowiedzi, i sprawdzić, czy prawidłowo przeanalizowaliśmy certyfikat.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8447:443 --name tls6 docker.io/mazurkatarzyna/tls-ex6:latest  
podman run -p 8447:443 --name tls6 docker.io/mazurkatarzyna/tls-ex6:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8447:443 --name tls6 ghcr.io/mazurkatarzynaumcs/tls-ex6:latest  
podman run -p 8447:443 --name tls6 ghcr.io/mazurkatarzynaumcs/tls-ex6:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8447>.

- (b) Wyślij request do endpointa <http://127.0.0.1:8447/cert> używając metody HTTP GET. Otrzymasz w odpowiedzi certyfikat TLS - zapisz go do pliku o nazwie `cert.pem`.
- (c) Z pobranego certyfikatu wyodrębnij klucz publiczny. Za pomocą metody HTTP POST, wyślij pod endpoint <http://127.0.0.1:8447/public> wygenerowany klucz publiczny (jako plik, `public_key`). W odpowiedzi serwer zwróci informację, czy klucz pasuje do certyfikatu.
- (d) Wyświetl certyfikat (pobrany z serwera plik `cert.pem`) w czytelnej postaci, przeanalizuj go i odpowiedz na poniższe pytania:
- Kiedy wygasa certyfikat (data)? (`expiry_date`)
 - Kiedy certyfikat został wydany (data)? (`issued_date`)
 - Kto jest wydawcą certyfikatu (Root CA)? (`issuer`)
 - Jaki algorytm został wykorzystany do wygenerowania klucza? (`key_algorithm`)
 - Jaki jest rozmiar klucza? (`key_size`)
 - Jaka jest główna domena certyfikatu? (`primary_domain`) Jakie są alternatywne domeny? (`san_domains`)
 - Do czego może być użyty klucz? (`key_usage`)
- (e) Za pomocą metody HTTP POST, wyślij pod endpoint <http://127.0.0.1:8447/validate> swoje odpowiedzi na powyższe pytania. Serwer przyzna punkty za każdą poprawną odpowiedź. (Maksymalnie można zdobyć 6 punktów).

Podpowiedź: Możesz skorzystać z endpointa <http://127.0.0.1:8447/apidocs>, aby sprawdzić, jak przesyłać swoje odpowiedzi do serwera.

6.6 Deszyfrowanie ruchu TLS przy użyciu klucza prywatnego serwera jest możliwe tylko wtedy, gdy połączenie wykorzystuje klasyczną wymianę kluczy RSA (tzw. `TLS_RSA`). W takim przypadku klient szyfruje pre-master secret kluczem publicznym serwera, a posiadanie klucza prywatnego pozwala ten sekret odszyfrować. Po jego odzyskaniu można wyliczyć master secret oraz klucze sesyjne, co umożliwia odtworzenie pełnej, zaszyfrowanej sesji TLS. W praktyce oznacza to, że mając klucz prywatny, da się odszyfrować ruch wszystkich klientów, którzy łączyli się z danym serwerem za pomocą RSA-based key exchange, o ile dysponujemy przechwyceniem ruchu. Nie działa to jednak w przypadku nowoczesnych połączeń używających ECDHE lub DHE, gdzie wykorzystywana jest tzw. forward secrecy. Tam klucz prywatny serwera nie wystarcza do deszyfrowania sesji, ponieważ każdy klient i serwer uzgadniają tymczasowe, jednorazowe klucze, które nie są nigdzie zapisane i których nie można odtworzyć z samego klucza prywatnego.

- (a) Twoim zadaniem jest odszyfrowanie ruchu sieciowego posiadając klucz prywatny serwera.
- (b) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8445:443 --name tls4 docker.io/mazurkatarzyna/tls-ex4:latest
podman run -p 8445:443 --name tls4 docker.io/mazurkatarzyna/tls-ex4:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8447:443 --name tls4 ghcr.io/mazurkatarzynaumcs/tls-ex4:latest
podman run -p 8447:443 --name tls4 ghcr.io/mazurkatarzynaumcs/tls-ex4:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8447>.

- (c) Otwórz w Wireshark'u plik `nettraffic1.pcapng`. Czy jesteś w stanie odczytać jego zawartość?
- (d) Dodaj do Wireshark'a klucz prywatny serwera:

```
Edit → Preferences → Protocols → TLS → RSA keys list
```

z wartościami:

```
0.0.0.0, 8443, http, /ścieżka/do/server.key
```

- (e) Sprawdź, czy ruch sieciowy został odszyfrowany.
- (f) Wykorzystaj serwer działający pod adresem <https://127.0.0.1:8447> i za pomocą Wireshark'a przechwyc ruch sieciowy do serwera. Następnie odszyfruj komunikację przy użyciu klucza prywatnego serwera.

6.7 Deszyfrowanie ruchu TLS przy użyciu pliku `SSLKEYLOGFILE` jest możliwe nawet wtedy, gdy połączenie korzysta z nowoczesnej wymiany kluczy ECDHE, zapewniającej forward secrecy. W przeciwieństwie do podejścia opartego na kluczu prywatnym serwera, tutaj to sam klient lub serwer zapisuje w specjalnym pliku tajne wartości (pre-master secret lub – w TLS 1.3 – bezpośrednio klucze sesyjne), zanim zostaną one użyte do szyfrowania ruchu. Dzięki temu można później, mając przechwycony ruch oraz ten plik, bez trudu odszyfrować pełną sesję TLS. Mechanizm działa niezależnie od tego, że ECDHE generuje tymczasowe, jednorazowe klucze – ponieważ `SSLKEYLOGFILE` przechwytyuje je w momencie tworzenia. W praktyce oznacza to, że można odszyfrować ruch każdego klienta, dla którego dostępny jest jego własny plik `SSLKEYLOGFILE`; nie daje on jednak dostępu do sesji innych klientów i nie pozwala podsłuchiwać ruchu pasywnie bez udziału klienta. Jest to więc metoda przydatna głównie do debugowania, testów lub analizy własnego ruchu, a nie narzędzie umożliwiające atak na cudze połączenia TLS.

- (a) Twoim zadaniem jest odszyfrowanie ruchu sieciowego przy użyciu pliku `SSLKEYLOGFILE`.
- (b) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8446:443 --name tls5 docker.io/mazurkatarzyna/tls-ex5:latest
podman run -p 8446:443 --name tls5 docker.io/mazurkatarzyna/tls-ex5:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8446:443 --name tls5 ghcr.io/mazurkatarzynaumcs/tls-ex5:latest  
podman run -p 8446:443 --name tls5 ghcr.io/mazurkatarzynaumcs/tls-ex5:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8446>.

- (c) Otwórz okno terminala i ustaw zmienną środowiskową `SSLKEYLOGFILE` za pomocą następującego polecenia, nie zamykaj okna terminala:

```
export SSLKEYLOGFILE=/tmp/sslkeys.log
```

- (d) Uruchom program **Wireshark** lub **tcpdump** i rozpocznij przechwytywanie pakietów (ruchu do serwera <https://127.0.0.1:8446>).
- (e) Uruchom przeglądarkę **Chrome** lub **Firefox** za pomocą okna terminala, które zostało użyte do ustawienia zmiennej środowiskowej. (Zmienna środowiskowa jest ustawiona tylko dla tej konkretnej sesji terminala):

```
firefox &
```

- (f) Użyj terminala, aby sprawdzić, czy plik `sslkeys.log` został utworzony, skorzystaj z poniższego polecenia:

```
file /tmp/sslkeys.log
```

- (g) Przejdź pod adres <https://127.0.0.1:8446> i wykonaj wszystkie czynności, które mają zostać przechwycone. Sprawdź w programie **Wireshark**, czy aktywność została prawidłowo przechwycona, a następnie zatrzymaj przechwytywanie.
- (h) W programie **Wireshark** przejdź do [Edytuj > Preferencje > Protokoły > TLS]. W polu Nazwa pliku dziennika (Pre)-Master-Secret wybierz plik `sslkey.log` utworzony wcześniej i kliknij OK. W **Wireshark**'u powinieneś zobaczyć odszyfrowany ruch.